

La philosophie DevOps

Avant DevOps

Traditionnellement, la gestion de projets a été dominée par la méthode **waterfall** (en cascade), qui impliquait une gestion **linéaire** du projet, étape par étape. Dans cette logique :

- **Tout d'abord**, les équipes de développement créent et améliorent des applications en respectant des contraintes de temps et de coût
- **Ensuite**, le code est transmis aux équipes d'exploitation, qui ont la responsabilité de le déployer en production tout contrôlant la stabilité et la qualité de l'application

Cependant, la transition entre ces deux phases peut s'avérer compliquée.

Problème de la méthode Waterfall

- On assiste au **cloisonnement** entre les équipes, qui ont des objectifs opposés : le développement du maximum de nouvelles fonctionnalités pour l'une, le maintien de la stabilité pour l'autre
- ➡ **"Wall of confusion"**, le dialogue entre les équipes est compliqué, on manque de confiance mutuelle
- Parfois, le code qui fonctionnait dans l'environnement de développement rencontre des problèmes en production

Objectif de DevOps

Le DevOps a pour objectif d'abattre ce "mur de confusion", et de favoriser la collaboration entre les équipes dans un **objectif partagé** :

“ livrer un produit de haute qualité, à moindre coût, dans des délais plus courts, qui réponde aux besoins du client. ”

Le DevOps : un métier ?

- Le DevOps **ne correspond pas à un poste**, et ne se limite pas à un seul rôle : toute personne prenant part aux différentes phases du cycle de vie d'une application doit adhérer à la culture DevOps
- Dans certaines organisations, quelques personnes ou équipes sont exclusivement dédiées à l'automatisation, à la définition des pratiques et à l'implémentation de pipelines CI/CD : ingénieur DevOps, spécialiste DevOps

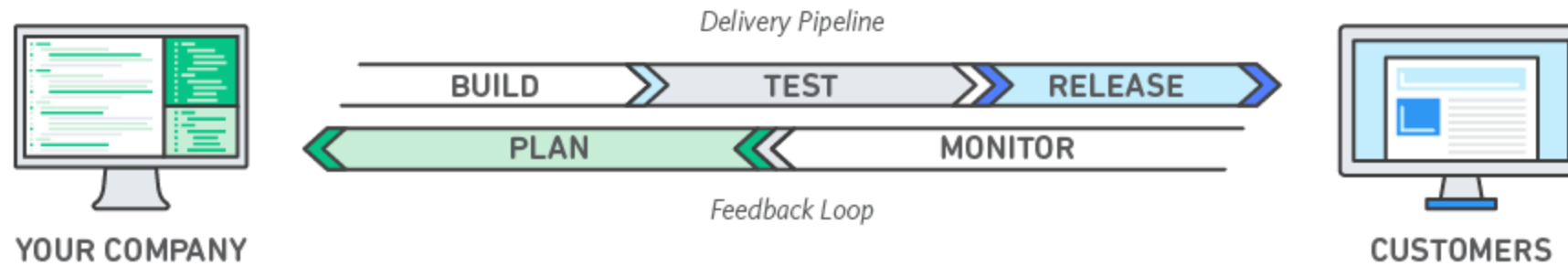
La naissance du mouvement DevOps

- Le mouvement se développe à partir de **2007** en Belgique, par les travaux de l'administrateur système **Patrick Debois** et du développeur **Andrew Shafer** qui forment l'Agile Administration System.
- Il s'inscrit dans un mouvement plus général de prise de conscience des **limites des modèles traditionnels** : l'essor du développement Lean, de l'Agile, de l'Extreme Programming, la création du métier de Site Reliability Engineer par Google en 2003.

Le modèle DevOps

- DevOps est une combinaison de **philosophies culturelles**, de **pratiques** et d'**outils** qui améliore la capacité d'une entreprise à livrer des applications et des services à un rythme élevé
- Il permet de faire **évoluer et d'optimiser les produits** plus rapidement que les entreprises utilisant des processus traditionnels de développement de logiciels et de gestion de l'infrastructure

Illustration du modèle



Fonctionnement de DevOps

- Dans un modèle DevOps, les équipes de développement et d'opérations ne sont plus isolées. Il arrive qu'elles soient fusionnées en une seule et même équipe
- Les ingénieurs qui la composent travaillent alors sur tout le cycle de vie d'une application : de la création à l'exploitation, en passant par les tests et le déploiement, et développent toute une gamme de compétences liées à différentes fonctions

La sécurité au coeur du DevOps

- Dans certains modèles DevOps, les équipes QA et de sécurité peuvent également s'intégrer étroitement au développement et aux opérations, ainsi qu'à l'ensemble du cycle de vie des applications
- Lorsque la sécurité est au cœur de l'activité d'une équipe DevOps, on parle parfois de [DevSecOps](#)

Pratiques et processus

- Les équipes DevOps utilisent des pratiques pour automatiser des processus qui étaient autrefois manuels et lents
- Elles exploitent une pile technologique et des outils qui les aident à faire fonctionner et à faire évoluer les applications de façon **rapide** et **fiable**
- Ces outils aident les ingénieurs à accomplir de façon autonome des tâches (par exemple, le déploiement de code) qui nécessiteraient normalement l'aide d'autres équipes, ce qui augmente encore davantage la productivité de l'équipe d'ingénieurs

Les avantages de DevOps (1/2)

- **Rapidité** : l'utilisation de micro-services et de la livraison continue permettent une mise à jour plus rapide
- **Livraison rapide** : l'intégration continue et la livraison continue automatique le processus de publication logiciel
- **Fiabilité** : l'intégration continue s'assure que le logiciel est fonctionnel et sûr, la supervision donne des indications sur les performances
- **Évolutivité** : l'IaC aide à gérer les environnements, du test à la prod

Les avantages de DevOps (2/2)

- **Collaboration améliorée** : collaboration de tous les acteurs impliqués dans le projet pour gagner en efficacité (réduction du délai de transfert entre les équipes de dev et d'infra)
- **Sécurité** : utilisation des politiques de conformité automatisées, des contrôles plus rigoureux et des techniques de gestion de la configuration

Les valeur DevOps (CALMS)

CALMS

On peut résumer les principes fondamentaux du DevOps à travers l'acronyme "CALMS" :

- **C**ulture
- **A**utomation
- **L**ean
- **M**easurement
- **S**haring

Culture

- DevOps n'est pas simplement un processus ou une approche différente du développement - **c'est un changement de culture**
- Tous les outils et l'automatisation du monde sont inutiles si les professionnels du développement et de l'IT/Ops ne travaillent pas ensemble. Car DevOps ne résout pas les problèmes d'outils. **Il résout les problèmes humains**
- Considérez DevOps comme une **évolution des équipes agiles**, à la différence que les opérations sont désormais incluses par défaut.

Automation

- L'automatisation permet d'éliminer les tâches manuelles répétitives, de mettre en place des processus reproductibles et de créer des systèmes fiables
- Les ordinateurs exécutent les tests plus rigoureusement et plus fidèlement que les humains
- Ces tests permettent de détecter plus rapidement les bogues et les failles de sécurité

Lean

- Lean sous-entend l'élimination des activités à faible valeur ajoutée et à la rapidité d'exécution, c'est-à-dire à la rapidité d'exécution et à l'agilité
- Il vise à maximiser la valeur client et à minimiser les gaspillages
- Identifier les tâches à forte valeur ajoutée et optimiser les autres est essentiel

Measurement

- La mesure permet d'évaluer les performances et d'identifier les domaines d'amélioration
- Suivre les métriques clés, comme le temps de déploiement ou la fréquence des mises à jour, est crucial

Sharing

- Le partage favorise la collaboration et la communication entre les équipes
- Partager les connaissances, les outils et les bonnes pratiques est essentiel pour réussir
- DevOps repose sur l'idée que les personnes qui construisent une application devraient être impliquées dans son expédition et son fonctionnement

Adopter un modèle DevOps

La philosophie culturelle DevOps (1/2)

- La transition vers DevOps implique un changement de culture et d'état d'esprit
- Pour simplifier, le DevOps consiste à **éliminer les obstacles** entre deux équipes traditionnellement isolées l'une de l'autre : **l'équipe de développement** et **l'équipe d'opérations**
- Avec DevOps, les deux équipes **travaillent en collaboration** pour **optimiser la productivité des développeurs** et la **fiabilité des opérations**

La philosophie culturelle DevOps (2/2)

- Elles ont à cœur de **communiquer fréquemment**, de gagner en efficacité et d'améliorer la qualité des services offerts aux clients
- Elles assument l'entière responsabilité de leurs services, et vont généralement **au-delà des rôles ou postes** traditionnellement définis en pensant **aux besoins de l'utilisateur final** et à comment les satisfaire

Les bonnes pratiques DevOps

- Il existe quelques pratiques clés pouvant aider les organisations à innover plus rapidement par le biais de l'**automatisation** et de la **rationalisation des processus de développement de logiciels** et de **gestion de l'infrastructure**
- La plupart de ces pratiques sont rendues possibles par l'utilisation **d'outils appropriés**

Mise à jour fréquentes

- Une pratique fondamentale consiste à réaliser des **mises à jour très fréquentes**, mais à **petite échelle**
- Ces mises à jour sont généralement de nature plus **incrémentielle**
- Le recours à des mises à jour fréquentes, mais à petite échelle, **limite les risques associés à chaque déploiement**
- Les équipes ont ainsi plus de **facilité à détecter les bogues** car pouvant identifier le dernier déploiement ayant provoqué l'erreur

Architecture micro-services

- L'architecture de microservices fragmente de grands systèmes complexes en des projets simples et indépendants
- Chaque service est associé à une **mission** ou **fonction spécifique** et exploité indépendamment des autres services et de l'application tout entière
- Cette architecture réduit les coûts de coordination liés aux mises à jour d'applications, et quand chaque service est tenu par de petites équipes agiles prenant pleine responsabilité de chaque service, les entreprises avancent plus rapidement

Intégration et déploiement continu

L'Intégration Continue (**CI**) :

- consiste à tester automatiquement les modifications de code
- La CI permet aux équipes d'apporter des changements au code plus fréquemment, améliorant ainsi la collaboration et la qualité finale du logiciel

Le déploiement continu (**CD**):

- automatiser la livraison d'applications
- harmonise la livraison du code entre les différents environnements

Infrastructure en tant que Code

- L'IaC (Infrastructure **as Code**) implique la mise en service et la gestion de l'infrastructure à l'aide de code et de techniques de développement de logiciels, notamment le contrôle des versions et l'intégration continue
- Les technologies cloud permettent de gérer l'infrastructure de manière programmatique et à n'importe quelle échelle, au lieu de devoir installer et configurer manuellement chaque ressource

Surveillance et journalisation

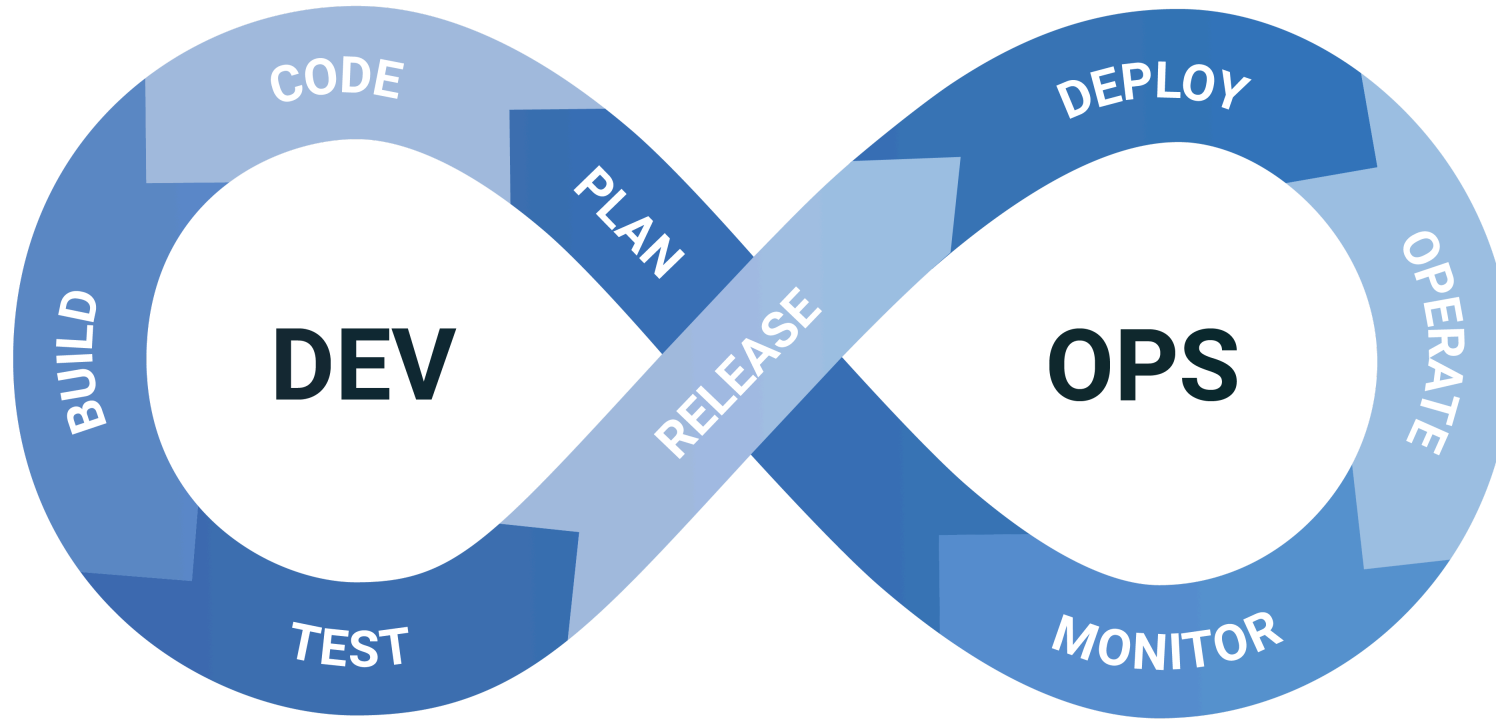
- Les entreprises surveillent les **métriques** et les **journaux** pour découvrir l'impact des performances de l'application et de l'infrastructure sur l'expérience de l'utilisateur final du produit
- La création d'alertes et **l'analyse en temps réel** de ces données aident également les entreprises à surveiller leurs services de manière plus proactive

Communication et collaboration

- Le recours aux outils DevOps et l'automatisation du processus de livraison des logiciels établit la collaboration en **rapprochant physiquement les flux de travail** entre les dev et les ops
- Les équipes instaurent des **normes culturelles** fortes autour du partage des informations et de la facilitation des communications, et ce par le biais d'**applications de messagerie**, de **systèmes de suivi des problèmes** ou des **projets** et de **wikis**

Le cycle DevOps

Le cycle DevOps



- La **planification continue** : le projet est divisé en de **multiples tâches**, avec des équipes concevant les fonctionnalités (Product Owners). Les backlogs sont utilisés pour regrouper l'ensemble des fonctionnalités
- Le **développement continu** : il implique des **versions logicielles fréquentes**, englobant l'écriture, le test, la révision et l'intégration du code
- L'**intégration continue** garantit que les **problèmes** sont résolus **au fur et à mesure**

- Le **déploiement continu** : les modifications sont déployées de manière automatique
- Les **tests continus** automatisés, à différentes étapes du cycle DevOps, vérifient l'intégrité du code et permettent de repérer les bugs au plus tôt
- Le **retour continu** consiste à collecter et traiter les commentaires des clients, même après le déploiement du produit, en vue de l'améliorer
- Les **opérations continues** automatisent le lancement du logiciel ainsi que ses futures mises à jour

Les outils DevOps

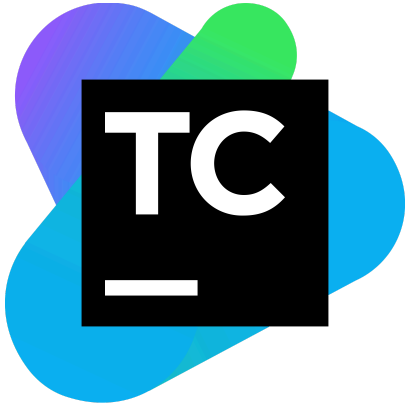
Les outils d'automatisation

L'automatisation joue un rôle fondamental dans l'approche DevOps.

Les outils de déploiement en continu:

- Scripts de déploiement automatisés, pipelines
- Jenkins, TeamCity, GitHub Actions : des exemples d'outils qui permettent de mettre en place un processus d'intégration, de déploiement et de livraison continus

TeamCity



TeamCity est un serveur d'intégration continue et de livraison continue développé par l'éditeur JetBrains.

Jenkins



- Jenkins est un outil open source de serveur d'automatisation écrit en java.
- Il s'interface avec des systèmes de gestion de versions tels que CVS, Git et Subversion, et exécute des projets basés sur Apache Ant et Apache Maven.

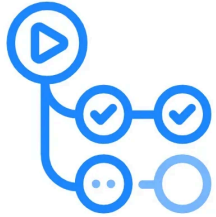
Jenkins

Jenkins est l'un des premiers serveurs d'intégration continue open source, et reste l'option la plus couramment utilisée aujourd'hui.

Ses avantages :

- Logiciel gratuit
- Facile à installer
- Plus de 1 000 plugins sont disponibles
- On peut créer et partager ses propres plugins

GitHub Actions



GitHub Actions



En lien direct avec GitHub, ce service permet d'exécuter automatiquement des workflows lorsque des événements se produisent dans le repository GitHub, y compris des tâches de CI/CD.

Qu'est-ce que l'intégration continue ?

- Le logiciel est reconstruit et testé à chaque modification apportée par un développeur
- On soumet **en continu** chaque changement de code à une batterie de tests **automatisés**
- Ainsi, les tests sont exécutés **parallèlement** au développement, et non à la fin. Les bugs sont détectés avant le déploiement.
- Types de tests : tests unitaires, d'intégration, end to end ...

Les conteneurs d'applications

Conteneurs d'applications

- Les conteneurs sont une technologie clé dans le paysage du cloud natif. Ils sont utilisés pour emballer et **isoler** les applications avec leurs **dépendances** entières, y compris le système d'exploitation, les bibliothèques système, les scripts, etc
- Cela permet d'assurer que l'application fonctionne de manière cohérente et fiable **dans n'importe quel environnement**, que ce soit en développement, en test ou en production

Conteneurs d'applications

- Un conteneur est **plus léger** qu'une machine virtuelle traditionnelle car il partage le système d'exploitation de l'hôte et n'a pas besoin de son propre système d'exploitation
- Cela rend les conteneurs très efficaces en termes d'utilisation des ressources système et de temps de démarrage

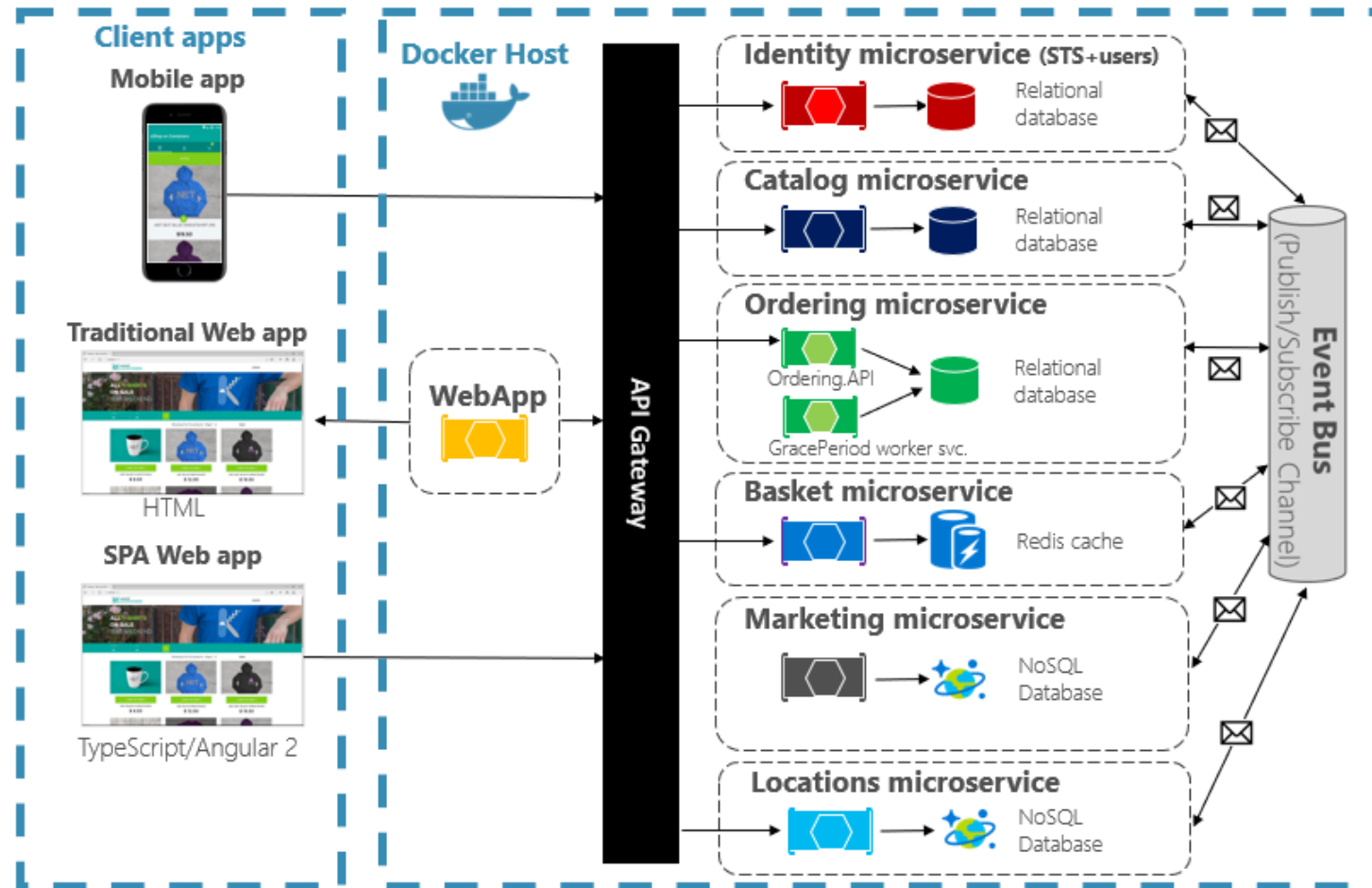
Conteneurs d'applications

- **Évolutivité** : Les conteneurs peuvent être démarrés et arrêtés rapidement, ce qui facilite leur mise à l'échelle en fonction de la demande. Si la demande pour une application augmente, plus de conteneurs peuvent être lancés pour gérer cette demande
- **Déploiement continu et livraison continue (CI/CD)** : Les conteneurs sont parfaits pour les pipelines CI/CD car ils permettent de déployer rapidement de nouvelles versions d'une application

Conteneurs d'applications

- **Isolation** : Chaque conteneur fonctionne de manière isolée. Cela signifie qu'un conteneur n'a pas d'effet sur les autres conteneurs et peut avoir ses propres configurations système et logicielles
- **Portabilité** : Les conteneurs garantissent que l'application fonctionne de manière identique dans tous les environnements. Cela permet de déplacer facilement les applications d'un système à un autre, ou d'un cloud à un autre

Conteneurs d'applications



Merci pour votre attention

Des questions ?